

What are some ways to improve visualization and organization on these graphs?

Here are several ways to improve the visualization and organization of your graphs:

1. Geospatial Map – Add Color-Coded Risk Levels

Right now your map plots every point the same color, making it hard to distinguish risk levels at a glance. Try mapping colors explicitly to each risk tier:

```
result_colors = {  
    "Risk 3 (Low)": "#2ca02c",  
    "Risk 2 (Medium)": "#d5c214",  
    "Risk 1 (High)": "#d62728",  
}
```

Also set `opacity=0.65` to handle overlapping points, and use `open-street-map` as your tile style for a clean, free basemap. Center it on Chicago with `{"lat": 41.8781, "lon": -87.6298}` and `zoom=10`.

2. Box Plot – Enforce a Logical Risk Ordering and Remove Noise

Your x-axis categories will render in arbitrary order unless you explicitly define it. Set:

```
risk_values = ["Risk 1 (High)", "Risk 2 (Medium)", "Risk 3 (Low)"]
```

and pass that as the `order` argument to `sns.boxplot()`. Also filter out any rows where `Risk` isn't one of those three values before plotting — unknown categories will distort your axis. Set `legend=False` since the color already communicates risk.

3. Heatmap – Normalize by Row So Risk Levels Are Comparable

If you just use raw counts, high-risk establishments will dominate every cell simply because there are more of them. Instead, convert counts to proportions within each risk group:

```
normalized_reasons = failure_reasons.div(failure_reasons.sum(axis=1), axis=0)
```

Then transpose the DataFrame so violation types are on the y-axis and risk levels are on the x-axis — it reads more naturally. Use `fmt=".1%"` in `sns.heatmap()` to annotate cells as percentages, and the `YlGnBu` colormap for a clean sequential look.

4. Bar Chart – Switch to Grouped Bars and Annotate Each Bar

Stacked bars make it hard to compare the middle segments across groups. Switch to `stacked=False` for side-by-side comparison. Then add percentage labels above each bar using `ax.patches`:

for `p` in `ax.patches`:

```
ax.annotate(f"{p.get_height():.1f}%",  
            (p.get_x() + p.get_width() / 2., p.get_height()),  
            ha="center", va="center", xytext=(0, 9),  
            textcoords="offset points", fontsize=9)
```

Use `normalize="index"` in `pd.crosstab()` and multiply by 100 so all bars represent percentages, and set `plt.ylim(0, 100)` so the scale is consistent.

How can I get started on an interactive geospatial map in python

Here's how to get started building an interactive geospatial map in Python:

1. Make Sure Your Coordinates Are Numeric

The first thing you need to do before touching any map library is clean your coordinate columns. Pandas reads CSVs as strings by default, so force them to numeric and let any bad values silently become NaN:

```
python
df["Latitude"] = pd.to_numeric(df["Latitude"], errors="coerce")
df["Longitude"] = pd.to_numeric(df["Longitude"], errors="coerce")
```

2. Build the Map with Plotly Express

`px.scatter_map` is the easiest way to get started. Point it at your coordinate columns, tell it what to color by, and add hover fields so the map is actually explorable:

```
python
fig = px.scatter_map(
    df,
    lat="Latitude", lon="Longitude", color="Risk",
    color_discrete_map=result_colors, hover_name="DBA Name",
    hover_data=["Address", "Facility Type", "Risk"], opacity=0.65,
    zoom=10, center={"lat": 41.8781, "lon": -87.6298},
    height=600, title="Chicago Food Inspection Map"
)
```

3. Set a Basemap and Tighten the Layout

`open-street-map` is free, requires no API key, and looks clean. The margin settings let the map fill its container with just enough room for the title:

```
python
fig.update_layout(
    mapbox_style="open-street-map",
    margin={"r":0, "t":40, "l":0, "b":0}
)
```

4. Show It

```
python
fig.show()
```

This opens the map in your browser as a fully interactive figure — you can zoom, pan, and hover over any point right away.